UCS 503 - Software Engineering

SmartLib: Library Seat Allocator

An IoT System for Real-Time Seat Monitoring and Management

Submitted by:

1024031135 Saksham Kansal
1024031118 Ishaan Gupta
1024031137 Sanshi

Computer Engineering Department (COE)

Submitted to:

Ms. Jhonsy Bansal

August 2026

Computer Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **2**
- 1.1 Background and Context . . . 2
  - 1.1.1 Problem Statement . . . 2
- 1.2 Project Objectives . . . 2
  - 1.2.1 Scope and Organisation of the Report . . . 3

2 Methodology and Design **4**
- 2.1 Requirements Analysis . . . 4
  - 2.1.1 Stakeholders . . . 4
  - 2.1.2 Functional Requirements . . . 4
  - 2.1.3 Non-Functional Requirements . . . 5
- 2.2 Software Development Process Model . . . 5
- 2.3 System Architecture . . . 6
  - 2.3.1 High-Level Design . . . 6
  - 2.3.2 Seating Layout . . . 6
  - 2.3.3 Technical Stack . . . 7
- 2.4 System Modeling Diagrams . . . 7
  - 2.4.1 Use Case Diagram . . . 8
  - 2.4.2 Data Flow Diagrams . . . 8
  - 2.4.3 Sequence Diagram . . . 9
  - 2.4.4 Seat State Machine . . . 10
  - 2.4.5 ESP32 Firmware Flowchart . . . 11
- 2.5 Implementation Details . . . 11
  - 2.5.1 Firmware (ESP32) . . . 11
  - 2.5.2 Cloud Layer and Web Application . . . 12

3 Results and Evaluation **13**
- 3.1 Testing Methodology . . . 13
- 3.2 Analysis . . . 13
  - 3.2.1 Limitations . . . 13

4 Conclusion **14**
- 4.1 Summary of Work . . . 14
- 4.2 Future Work and Recommendations . . . 14

Bibliography **15**

Chapter 1

Introduction

1.1 Background and Context

University libraries are busiest during exams and assignment deadlines. At these times, users have to walk through the reading halls to find a free seat. Gate counters show how many people are inside, but not *where* the free seats are. This wastes time and creates noise that disturbs people who are studying.

Low-cost Wi-Fi boards, simple sensors and real-time cloud databases now make it possible to track every seat and show its status on any phone or screen within a second.

1.1.1 Problem Statement

Finding a seat today has three main problems, listed in Table 1.1.

Problem	**Description**	**What is needed**
Manual search	Users walk across floors and aisles at busy times, which wastes time and creates noise.	A way to see free seats before walking to the hall.
No seat-level view	Headcount boards at the gate show only a total count, not which desks are free.	Status of each seat instead of one building-wide number.
No data for staff	Staff have no live or past usage data to spread users across halls or plan facilities such as cooling.	Live and past usage reports for planning.

Table 1.1: Main problems and what is needed

1.2 Project Objectives

1. **Detection:** Detect whether each seat is in use with low-cost IR sensors.
2. **Noise filtering:** Ignore short disturbances, such as a passing hand, so a seat's status does not flicker.
3. **Transmission:** Send seat status to the cloud over Wi-Fi with little delay.

4. **User view:** Show a live seat map and seat counts on phones and on a screen at the entrance.

5. **Admin view:** Give staff a dashboard with usage, session time and busy hours.

1.2.1 Scope and Organisation of the Report

The system covers sensing, sending, storing and displaying seat status for one library zone. It only detects whether a seat is in use. It does not take pictures, identify people or handle seat bookings. The prototype shows the full path from a sensor to a web browser, and more seats or zones can be added with more sensor nodes and database entries.

Chapter 2 covers the requirements, development model, design, diagrams and implementation. Chapter 3 gives the test plan and limitations, and Chapter 4 concludes with future work.

Chapter 2

Methodology and Design

2.1 Requirements Analysis

2.1.1 Stakeholders

- **Users:** Want to know which seats are free and how full the library is.
- **Library staff / administrators:** Want live and past usage data to manage the library.
- **Sensor node (system actor):** Checks each seat and sends its status.

2.1.2 Functional Requirements

ID	Requirement
FR1	Detect whether a person is at each seat using an IR sensor.
FR2	Change a seat's status only after the new reading stays the same for 3 seconds (debounce).
FR3	The ESP32 sends seat status as JSON over Wi-Fi to the cloud database.
FR4	The database stores the latest status and updates all screens without a page refresh.
FR5	The user dashboard shows a seat map: free seats in green, taken seats in red.
FR6	The dashboards show total, taken and free seats, and the percentage in use.
FR7	The entrance screen shows live seat availability for the zone.
FR8	The admin dashboard shows average session time, busiest hour, weekly trend, hourly usage and the taken/free split.

Table 2.1: Functional requirements

2.1.3 Non-Functional Requirements

ID	Attribute	Requirement
NFR1	Performance	Screens update within one second of a database change (not counting the 3-second debounce).
NFR2	Reliability	Short disturbances must not make seat colours flicker.
NFR3	Usability	The user interface works well on a phone browser.
NFR4	Scalability	New seats and zones can be added without changing the design.
NFR5	Cost	Uses cheap, easily available hardware.
NFR6	Privacy	No images or personal data; only “in use” or “free” is stored.

Table 2.2: Non-functional requirements

2.2 Software Development Process Model

The project follows the *Incremental Development Model* [Som16, PM19]. The requirements in Section 2.1 are set first. The system is then built in parts (increments). Each part is designed, built and tested, then joined to the parts already finished, so every stage ends with something that works (Figure 2.1).

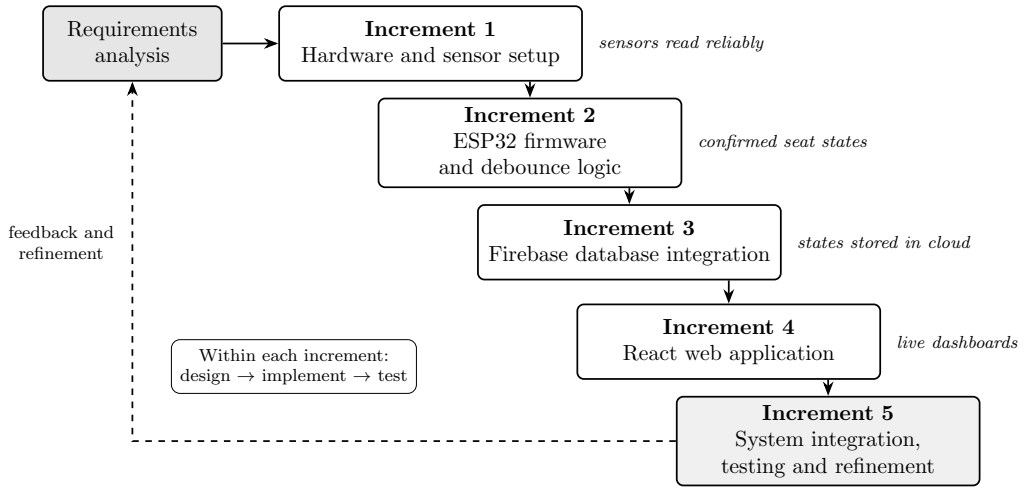


Figure 2.1: Incremental development model for the Library Seat Allocator

Increment 1 fits the IR sensors to the ESP32 and checks their output at each seat. Increment 2 adds sensor reading and the 3-second debounce (FR1, FR2). Increment 3 sends the seat status to Firebase as JSON (FR3, FR4). Increment 4 builds the user

dashboard, admin dashboard and entrance screen (FR5–FR8). Increment 5 joins everything and tests it with the cases in Chapter 3; any problems are fixed in the earlier parts.

Why this model. The project has hardware, firmware, a cloud database and a web app, and each part depends on the one before it. Building them one at a time lets each part be tested on its own, so a fault is easy to locate. Each increment also gives a working result that can be shown, and later parts can change without starting over.

2.3 System Architecture

2.3.1 High-Level Design

The system has three layers (Figure 2.2): the *edge layer* senses the seats, the *cloud layer* stores and shares their status, and the *presentation layer* shows it to users and staff.

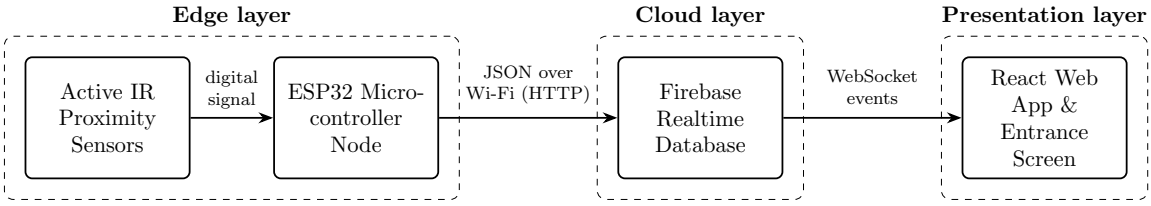


Figure 2.2: End-to-end system architecture

The IR sensors send signals to the ESP32 [Esp], which sends the confirmed seat status as JSON over Wi-Fi to the Firebase Realtime Database [Goo]. The database then pushes each change to the React web app and the entrance screen [Met].

2.3.2 Seating Layout

Figure 2.3 shows the seat arrangement. Each seat has an IR sensor at the table edge, facing the chair. The sensors are wired to a controller linked to the ESP32. In the figure, a free seat is white and a taken seat is dark; the dashboards use green and red.

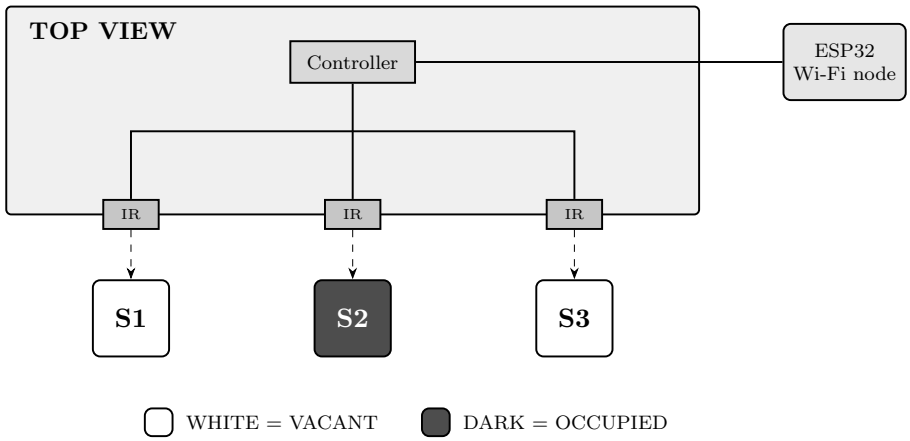


Figure 2.3: Seating layout (top view): per-seat IR sensors wired to a controller and an ESP32 node

2.3.3 Technical Stack

- **Hardware:** ESP32 development board [Esp]; one IR proximity sensor per seat (GPIO 27 and GPIO 28 in the prototype); two push buttons on GPIO 18 and GPIO 19; a 5 V power supply; perforated board and jumper wires.
- **Framework/Language:** C++ with the Arduino framework for the ESP32 firmware; React with JavaScript for the web app [Met].
- **Database:** Firebase Realtime Database [Goo]. Seat status is kept as JSON under `library/zone1`, with one entry per seat (`s1`, `s2`, ...) holding a true/false `occupied` value.
- **Libraries:** the ESP32 `WiFi` and `HTTPClient` libraries for sending data; the Firebase SDK in the web app for live updates; SVG for drawing the seat map.
- **Tools:** an Arduino-compatible IDE to build and upload the firmware; a web browser on a laptop and a phone for testing the dashboards.

2.4 System Modeling Diagrams

Each diagram shows the system from a different angle: who uses it (use case), how data moves (DFDs), what happens over time (sequence diagram and state machine) and the order of steps in the firmware (flowchart).

2.4.1 Use Case Diagram

Figure 2.4 shows who uses the system and how [BRJ05]. Users view the seat map and availability, staff view the reports, and the sensor node detects seats and sends their status.

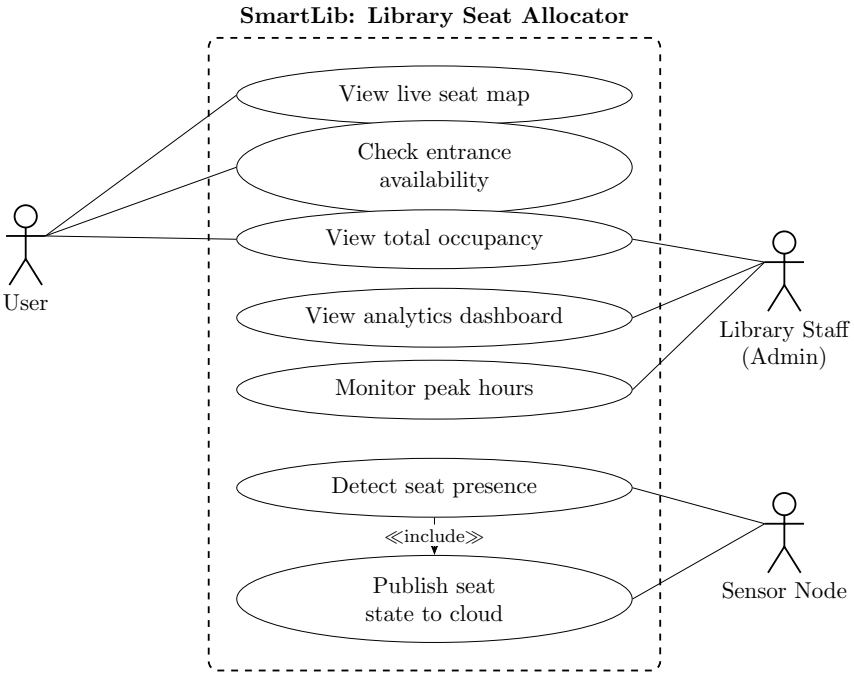


Figure 2.4: Use case diagram

2.4.2 Data Flow Diagrams

The level-0 diagram (Figure 2.5) shows the whole system as one process. The level-1 diagram (Figure 2.6) splits it into four processes and one data store.

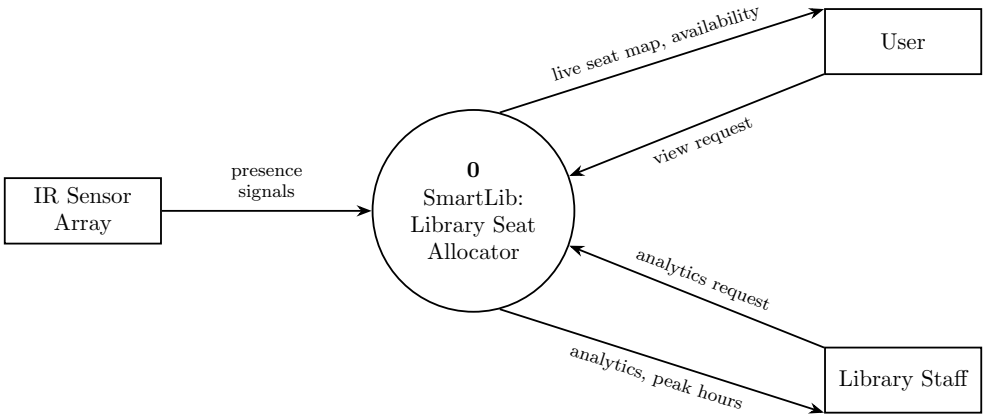


Figure 2.5: Data flow diagram – level 0 (context diagram)

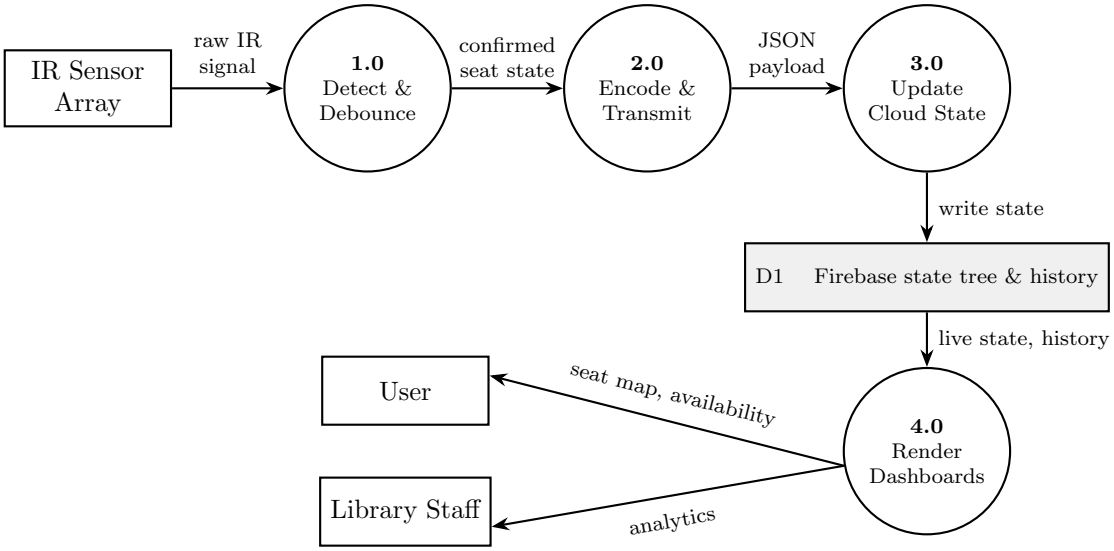


Figure 2.6: Data flow diagram – level 1

2.4.3 Sequence Diagram

Figure 2.7 follows one event, from a user sitting down to the seat turning red on every dashboard. The ESP32 sends the update only after the reading stays the same for 3 seconds.

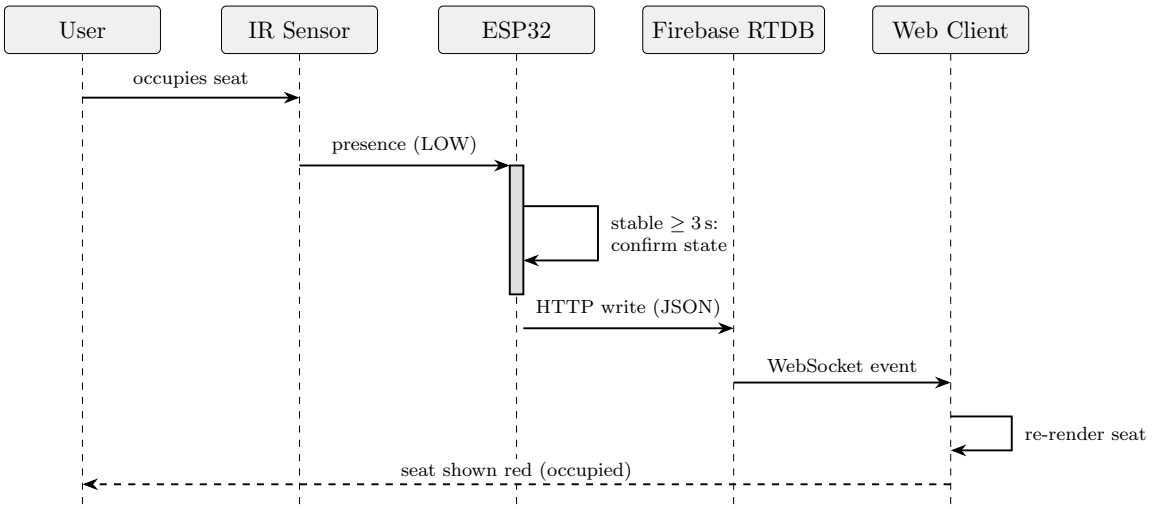


Figure 2.7: Sequence diagram for a seat-occupancy event, including the 3-second debounce

2.4.4 Seat State Machine

Each seat follows the four states in Figure 2.8. VACANT and OCCUPIED are the states sent to the database. In the two PENDING states the firmware waits to see if the new reading holds. If it holds for 3 seconds, the status changes and is sent. If it changes back sooner (dashed arrows), the seat keeps its old status and nothing is sent.

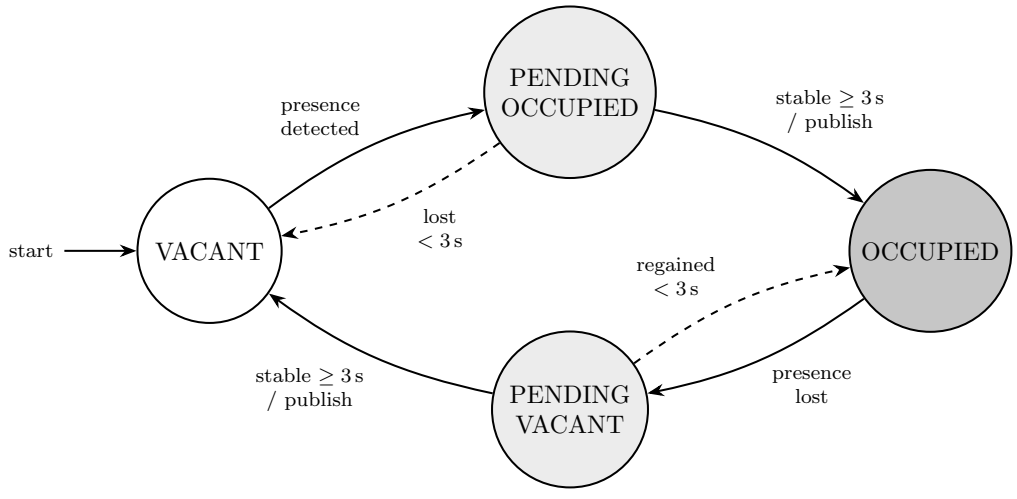


Figure 2.8: Seat state machine with 3-second debounce

2.4.5 ESP32 Firmware Flowchart

Figure 2.9 shows the order of steps in the firmware. After start-up, the loop reads each sensor, checks whether the reading has changed and held for 3 seconds, and sends an update only when a seat's status changes. The state machine describes one seat; the flowchart describes the program.

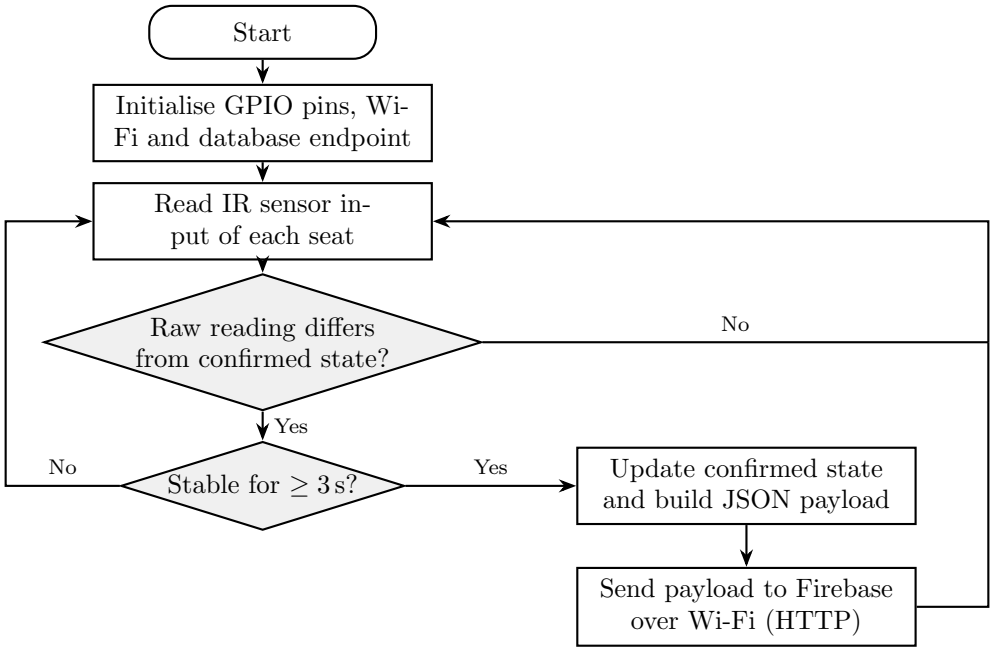


Figure 2.9: Firmware main-loop flowchart

2.5 Implementation Details

2.5.1 Firmware (ESP32)

The firmware reads the IR pins, applies the 3-second debounce to each seat and sends the confirmed status as JSON. Listing 2.1 shows this logic; pin numbers match Section 2.3.3.

Listing 2.1: Illustrative firmware logic for debounce and publishing

```
constexpr uint32_t DEBOUNCE_MS = 3000;   // 3-second debounce window
struct Seat { uint8_t pin; bool confirmed, candidate; uint32_t changedAt; };
Seat seats[2] = { {27, false, false, 0}, {28, false, false, 0} };

void publishState() {                     // build JSON and write once
  String json = "{";
  for (int i = 0; i < 2; ++i) {
    json += "\"s" + String(i + 1) + "\":{\"occupied\":";
    json += seats[i].confirmed ? "true}" : "false}";
    if (i < 1) json += ",";
  }
  json += "}";
  HTTPClient http;  http.begin(DB_URL);
  http.PUT(json);   http.end();           // clients notified by the database
}
void loop() {
  bool dirty = false;
  for (auto &s : seats) {
    bool raw = (digitalRead(s.pin) == LOW);         // active-low IR module
    if (raw != s.candidate) { s.candidate = raw; s.changedAt = millis(); }
    if (s.candidate != s.confirmed && millis() - s.changedAt >= DEBOUNCE_MS) {
      s.confirmed = s.candidate;  dirty = true;
    }
  }
  if (dirty) publishState();
}
```

2.5.2 Cloud Layer and Web Application

The Firebase Realtime Database holds the seat status described in Section 2.3.3. The web app listens to the database and receives each change over an open WebSocket connection, so it never needs to ask for updates.

The web app, called *SmartLib*, is built with React [Met]. It has a *User* view and an *Admin* view in the navigation bar. Each seat on the map is an SVG shape whose colour follows its `occupied` value, so it updates as soon as the database changes.

User dashboard. Cards show total, taken and free seats and the percentage in use (for example: 4 seats, 2 taken, 2 free, 50%). A seat map shows seats S01–S04 around a table, with free seats in green and taken seats in red.

Admin dashboard. Cards show total seats, taken seats, average session time and the busiest hour. Charts show the weekly trend, hourly usage and the taken/free split, to help staff plan.

Entrance screen. The same app shows live seat availability at the library gate.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

The planned tests in Table 3.1 check the requirements from Chapter 2.

ID	Scenario	Expected outcome	Req.
T1	A person sits in a free seat	After 3 s the seat turns red on all dashboards	FR1, FR2, FR5
T2	The person leaves the seat	After 3 s the seat turns green	FR1, FR2, FR5
T3	A hand or bag passes the sensor for less than 3 s	No change is sent	FR2, NFR2
T4	Two screens are open at once	Both update together without a refresh	FR4
T5	Seat counts are checked	Taken + free equals total seats; the percentage is correct	FR6
T6	Wi-Fi drops and comes back	The node sends updates again and dashboards show the current status	NFR1
T7	The entrance screen is opened at the gate	It matches the user dashboard	FR7
T8	The admin dashboard is opened	Session time, busiest hour and charts are shown	FR8

Table 3.1: Test cases and expected outcomes

3.2 Analysis

The prototype shows the full path: IR sensor to ESP32, ESP32 to the cloud database, and database to live browser dashboards. Sensing each seat fixes the main weakness of gate counters, and the 3-second debounce keeps seat colours steady.

3.2.1 Limitations

An IR sensor detects any object, not only a person, so a bag left on a desk shows the seat as taken. The system needs library Wi-Fi and the cloud service, and larger set-ups need more nodes, careful wiring and power planning, and sensor tuning.

Chapter 4

Conclusion

4.1 Summary of Work

This project builds SmartLib, a real-time Library Seat Allocator, from low-cost IoT hardware and a cloud database. It solves the three problems found earlier: users can see free seats before walking in, each desk's status is shown instead of a total count, and staff get live and past usage data.

4.2 Future Work and Recommendations

1. Add a second type of sensor to tell a person apart from an object.
2. Add seat booking with time-limited holds.
3. Use past data to predict busy hours.
4. Use a lighter messaging method such as MQTT, and battery or low-power operation.
5. Cover more zones and floors with a campus-wide map.
6. Send alerts when a chosen seat or zone becomes free.

Bibliography

[BRJ05] Grady Booch, James Rumbaugh, and Ivar Jacobson. *The Unified Modeling Language User Guide*. 2nd ed. Addison-Wesley, 2005.

[Esp] Espressif Systems. *ESP32 Series Datasheet*. Espressif Systems. `https://www.espressif.com`.

[Goo] Google. *Firebase Realtime Database Documentation*. `https://firebase.google.com/docs/database`.

[Met] Meta Open Source. *React Documentation*. `https://react.dev`.

[PM19] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. 9th ed. McGraw-Hill, 2019.

[Som16] Ian Sommerville. *Software Engineering*. 10th ed. Pearson, 2016.